Week 1 Training Plan: Python Revision & OOP

Day 1: Python Fundamentals Refresher

Topics

- Variables, data types (int, float, str, bool, None) and type casting
- Operators: arithmetic, comparison, logical, assignment, `is` vs `==`
- String formatting: f-strings (primary), `.format()`
- Conditionals: `if`/`elif`/`else`, nested conditions, ternary expressions
- Loops: `for`, `while`, `range()`, `break`/`continue`/`pass`, loop `else`
- `input()` / `print()` with `sep` and `end`

Practice Questions

1. Write a program to check whether a number is prime.
2. Reverse a string without using slicing or a built-in reverse function.
3. Print the Fibonacci series up to n terms.
4. FizzBuzz: print numbers 1 to 100, replacing multiples of 3 with “Fizz”, multiples of 5 with “Buzz”, and multiples of both with “FizzBuzz”.
5. Check if a given string is a palindrome.
6. Given a list of numbers, find the largest and smallest without using `min()`/`max()`.

Day 2: Data Structures

Topics

- Lists: indexing, slicing, common methods (`append`, `extend`, `insert`, `remove`, `pop`, `sort`), list comprehensions
- Tuples: immutability, packing/unpacking, when to prefer them over lists
- Dictionaries: methods (`get`, `keys`, `values`, `items`, `update`), dict comprehensions
- Sets: union, intersection, difference, symmetric difference, common use cases
- Strings: `split`, `join`, `strip`, `replace`, `find`, `count`, immutability

Practice Questions

1. Use a list comprehension to extract all even numbers from a list in one line.
2. Count the frequency of each word in a sentence using a dictionary.
3. Find duplicate elements in a list using a set.
4. Merge two dictionaries; if a key exists in both, sum the values.
5. Given a sentence, count the vowels and consonants separately.
6. Write a function that returns the most frequent element in a list.

Day 3: Functions & Functional Tools

Topics

- Function basics: `def`, parameters, return values, default arguments
- `*args` and `**kwargs`
- Lambda functions
- `map()`, `filter()`, `zip()`, `enumerate()`, `sorted()` with a `key`
- Scope: local vs global, the `global` keyword
- Recursion (factorial, Fibonacci)
- Modules: `import`, `from ... import`, writing your own module

Practice Questions

1. Write a function that accepts any number of arguments and returns their sum (use `*args`).
2. Use `map()` and a `lambda` to square every number in a list.
3. Use `filter()` to extract all strings longer than 5 characters from a list.
4. Write a recursive function to calculate the factorial of a number.
5. Sort a list of dictionaries (e.g., students with `name` and `marks`) by `marks` using `sorted()` and a `key` function.
6. Write a function with a default argument, and add a comment explaining why mutable default arguments (like a list) are risky in Python.

Day 4: Object-Oriented Programming, Part 1

Topics

- Classes and objects: defining a class, creating instances
- The `__init__` constructor and `self`
- Instance variables vs class variables
- Instance methods vs `@classmethod` vs `@staticmethod`
- Encapsulation: public, `_protected`, `__private` conventions
- `@property` for controlled attribute access (getters/setters)

Practice Questions

1. Create a `Car` class with attributes (`brand`, `model`, `year`) and a method that displays the car's info.
2. Create a `BankAccount` class with `deposit()` and `withdraw()` methods; prevent withdrawals beyond the balance.
3. Demonstrate the difference between instance and class variables: track how many objects of a class have been created.
4. Write a class with a private attribute, exposing it safely through `@property`.

5. Add a `@classmethod` to a `Person` class that works as an alternate constructor, e.g. `Person.from_birth_year(name, year)`.

Day 5: OOP Part 2 + Most-Used Practical Python

Topics

- Inheritance: single and multiple inheritance, `super()`
- Polymorphism: method overriding, duck typing
- Abstraction: abstract base classes (`abc` module)
- Dunder/magic methods: `__str__`, `__repr__`, `__len__`, `__eq__`, `__add__`
- Exception handling: `try`/`except`/`else`/`finally`, raising custom exceptions
- File handling: reading/writing files, the `with` statement (context managers)
- Working with JSON: `json.load`/`loads`, `json.dump`/`dumps`
- Quick-hit practical extras: virtual environments (`venv`), `pip` + `requirements.txt`, basic type hints

Practice Questions

1. Create a base class `Animal` with a `speak()` method, and `Dog`/`Cat` subclasses that override it.
2. Define a custom exception `InsufficientBalanceError` and raise it inside the `BankAccount` class from Day 4.
3. Write a program that reads a text file and reports the number of lines, words, and characters.
4. Write a program that saves a list of dictionaries (student records) to a JSON file, then loads them back.
5. Implement `__str__` and `__repr__` for a `Point(x, y)` class so printing an instance gives readable output.
6. Write a `try`/`except` block that gracefully handles at least three different exception types (`ValueError`, `ZeroDivisionError`, `FileNotFoundError`).

End-of-Week Capstone: Library Management System (CLI)

A small project to pull together everything from the week. Suggested scope:

- `Book` class: `title`, `author`, `isbn`, `copies_available`
- `Member` class: `name`, `member_id`, `borrowed_books`
- `Library` class: manages books and members, with `add_book()`, `issue_book()`, `return_book()`, `search_book()`
- Persist all data to a JSON file so it survives between runs
- Handle edge cases with proper exceptions: book not found, no copies available, member already holding that book, returning a book they never borrowed